15 Commonly Used String Methods in Java

Name : Mahammad Ashiya

Roll No : A25126552231

1. length() — studentName

Objective

Objective: To understand and demonstrate the Java String method **length()** using a simple example.

Task Description

Task Description: Write and execute the class **StringLengthDemo** to demonstrate that length() finds the number of characters in a string.

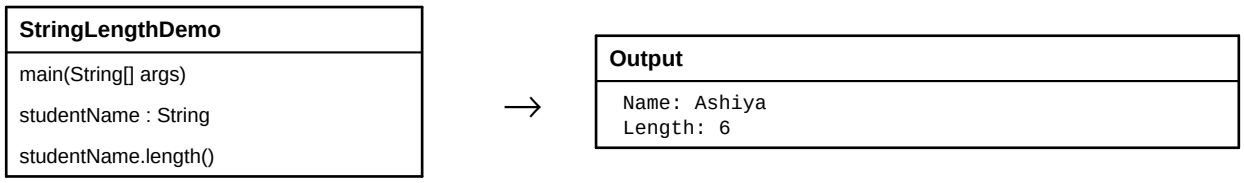
Step-wise Execution

1. Create a String variable named studentName.
2. Store the value "Ashiya" in it.
3. Call the length() method using studentName.
4. Display the returned number of characters.

Algorithm / Logic

1. Start.
2. Initialize studentName with "Ashiya".
3. Find the length using studentName.length().
4. Print the name and length.
5. Stop.

UML Diagram



Program Code

```
class StringLengthDemo {
    public static void main(String[] args) {
        String studentName = "Ashiya";

        System.out.println("Name: " + studentName);
        System.out.println("Length: " + studentName.length());
    }
}
```

Code Explanation

1. `String studentName = "Ashiya";` creates a String variable containing the name.
2. `studentName.length()` returns the total number of characters in the string.
3. `System.out.println()` displays the name and calculated length.

Output Screenshot

2. charAt() — courseName

Objective

Objective: To understand and demonstrate the Java String method **charAt()** using a simple example.

Task Description

Task Description: Write and execute the class **CharacterAtDemo** to demonstrate that charAt() returns the character at a specified index position.

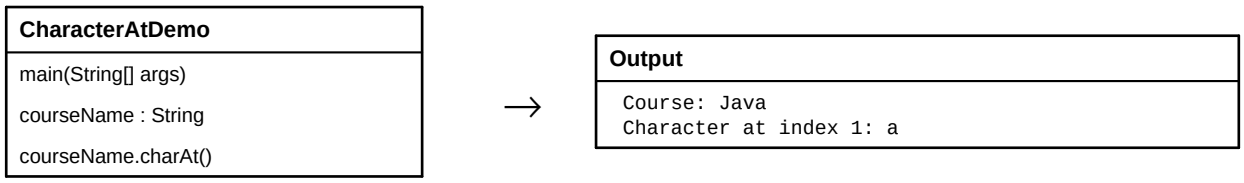
Step-wise Execution

1. Create the courseName string.
2. Store "Java" in the variable.
3. Call charAt(1) to access the character at index 1.
4. Print the course name and selected character.

Algorithm / Logic

1. Start.
2. Initialize courseName with "Java".
3. Pass index 1 to charAt().
4. Display the returned character.
5. Stop.

UML Diagram



Program Code

```
class CharacterAtDemo {
    public static void main(String[] args) {
        String courseName = "Java";

        System.out.println("Course: " + courseName);
        System.out.println("Character at index 1: " + courseName.charAt(1));
    }
}
```

Code Explanation

1. `courseName` stores the string "Java".
2. String indexing starts from 0, so index 1 refers to the second character.
3. `courseName.charAt(1)` returns the character `a`.

Output Screenshot

3. toUpperCase() — cityName

Objective

Objective: To understand and demonstrate the Java String method **toUpperCase()** using a simple example.

Task Description

Task Description: Write and execute the class **UpperCaseDemo** to demonstrate that toUpperCase() converts all letters in a string to uppercase.

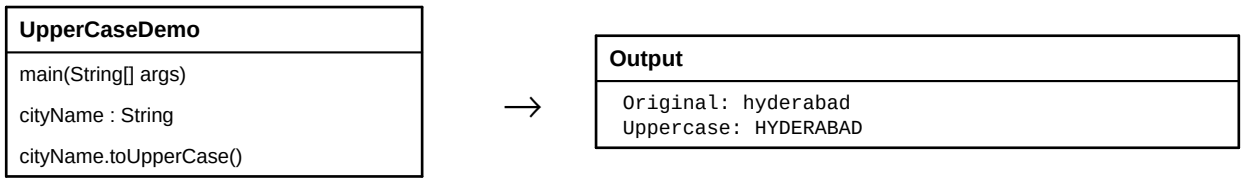
Step-wise Execution

1. Create cityName and assign "hyderabad".
2. Print the original value.
3. Call cityName.toUpperCase().
4. Print the converted uppercase value.

Algorithm / Logic

1. Start.
2. Initialize cityName.
3. Apply toUpperCase() to cityName.
4. Display original and uppercase strings.
5. Stop.

UML Diagram



Program Code

```
class UpperCaseDemo {
    public static void main(String[] args) {
        String cityName = "hyderabad";

        System.out.println("Original: " + cityName);
        System.out.println("Uppercase: " + cityName.toUpperCase());
    }
}
```

Code Explanation

1. `cityName` stores the lowercase text "hyderabad".
2. `toUpperCase()` creates and returns an uppercase version of the string.
3. The original string is unchanged; the converted value is printed.

Output Screenshot

4. toLowerCase() — collegeName

Objective

Objective: To understand and demonstrate the Java String method **toLowerCase()** using a simple example.

Task Description

Task Description: Write and execute the class **LowerCaseDemo** to demonstrate that toLowerCase() converts all letters in a string to lowercase.

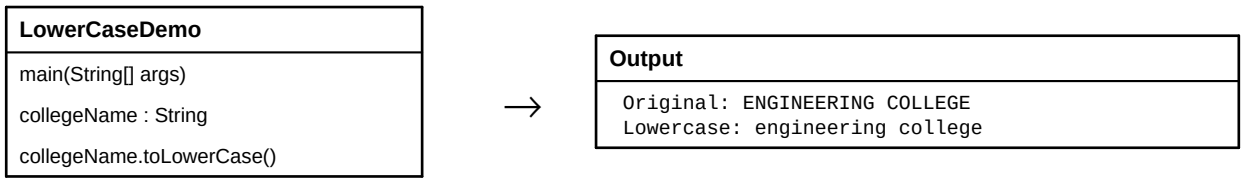
Step-wise Execution

1. Create collegeName.
2. Store "ENGINEERING COLLEGE".
3. Call toLowerCase() on the string.
4. Print the original and lowercase values.

Algorithm / Logic

1. Start.
2. Initialize collegeName.
3. Apply toLowerCase().
4. Display both strings.
5. Stop.

UML Diagram



Program Code

```
class LowerCaseDemo {
    public static void main(String[] args) {
        String collegeName = "ENGINEERING COLLEGE";

        System.out.println("Original: " + collegeName);
        System.out.println("Lowercase: " + collegeName.toLowerCase());
    }
}
```

Code Explanation

1. `collegeName` stores uppercase text.
2. `toLowerCase()` returns a lowercase version of the string.
3. The original value remains unchanged and both values are displayed.

Output Screenshot

5. equals() — firstWord

Objective

Objective: To understand and demonstrate the Java String method **equals()** using a simple example.

Task Description

Task Description: Write and execute the class **StringEqualsDemo** to demonstrate that equals() compares two strings and returns true when their contents are equal.

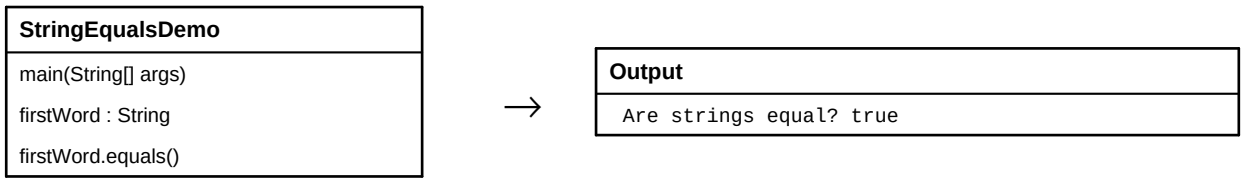
Step-wise Execution

1. Create two String variables.
2. Assign "Java" to both variables.
3. Call firstWord.equals(secondWord).
4. Print the Boolean result.

Algorithm / Logic

1. Start.
2. Initialize firstWord and secondWord.
3. Compare their contents using equals().
4. Print true or false.
5. Stop.

UML Diagram



Program Code

```
class StringEqualsDemo {
    public static void main(String[] args) {
        String firstWord = "Java";
        String secondWord = "Java";

        System.out.println("Are strings equal? "
                + firstWord.equals(secondWord));
    }
}
```

Code Explanation

1. `firstWord` and `secondWord` both contain "Java".
2. `equals()` compares the contents of the two strings.
3. Because the contents are the same, the method returns `true`.

Output Screenshot

6. equalsIgnoreCase() — userInput

Objective

Objective: To understand and demonstrate the Java String method **equalsIgnoreCase()** using a simple example.

Task Description

Task Description: Write and execute the class **IgnoreCaseDemo** to demonstrate that equalsIgnoreCase() compares two strings without considering uppercase or lowercase differences.

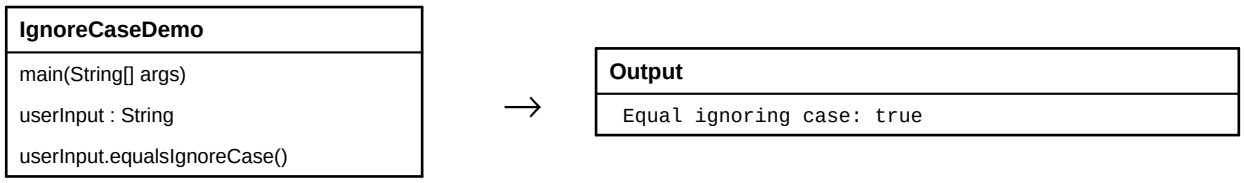
Step-wise Execution

1. Create userInput and correctWord.
2. Assign uppercase and lowercase forms of the same word.
3. Call equalsIgnoreCase().
4. Display the Boolean result.

Algorithm / Logic

1. Start.
2. Initialize the two strings.
3. Compare them while ignoring case.
4. Print the comparison result.
5. Stop.

UML Diagram



Program Code

```
class IgnoreCaseDemo {
    public static void main(String[] args) {
        String userInput = "HELLO";
        String correctWord = "hello";

        System.out.println("Equal ignoring case: "
                + userInput.equalsIgnoreCase(correctWord));
    }
}
```

Code Explanation

1. `userInput` contains "HELLO" and `correctWord` contains "hello".
2. `equalsIgnoreCase()` compares the strings without considering letter case.
3. The strings represent the same letters, so the result is `true`.

Output Screenshot

7. contains() — messageText

Objective

Objective: To understand and demonstrate the Java String method **contains()** using a simple example.

Task Description

Task Description: Write and execute the class **StringContainsDemo** to demonstrate that contains() checks whether a specified sequence of characters exists in a string.

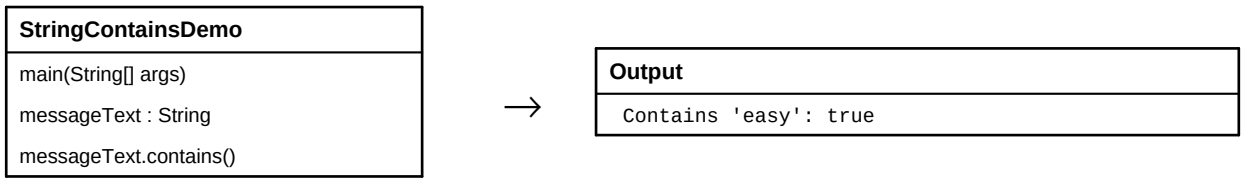
Step-wise Execution

1. Create messageText.
2. Store the sentence in the variable.
3. Search for the word "easy" using contains().
4. Print the Boolean result.

Algorithm / Logic

1. Start.
2. Initialize messageText.
3. Call contains("easy").
4. Display true if the text is found.
5. Stop.

UML Diagram



Program Code

```
class StringContainsDemo {
    public static void main(String[] args) {
        String messageText = "Java is very easy";

        System.out.println("Contains 'easy': "
                + messageText.contains("easy"));
    }
}
```

Code Explanation

1. `messageText` stores the sentence "Java is very easy".
2. `contains("easy")` searches for the text `easy` inside the sentence.
3. Since the text is present, the method returns `true`.

Output Screenshot

8. substring() — programTitle

Objective

Objective: To understand and demonstrate the Java String method **substring()** using a simple example.

Task Description

Task Description: Write and execute the class **SubstringDemo** to demonstrate that substring() extracts a part of a string using index positions.

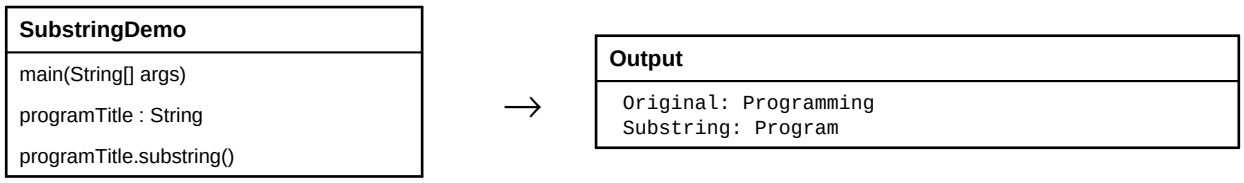
Step-wise Execution

1. Create programTitle.
2. Store "Programming".
3. Call substring(0, 7).
4. Print the extracted substring.

Algorithm / Logic

1. Start.
2. Initialize programTitle.
3. Specify start index 0 and end index 7.
4. Extract and display the substring.
5. Stop.

UML Diagram



Program Code

```
class SubstringDemo {
    public static void main(String[] args) {
        String programTitle = "Programming";

        System.out.println("Original: " + programTitle);
        System.out.println("Substring: "
                + programTitle.substring(0, 7));
    }
}
```

Code Explanation

1. `programTitle` stores the word "Programming".
2. `substring(0, 7)` starts at index 0 and stops before index 7.
3. Therefore, the characters at indexes 0 through 6 form "Program".

Output Screenshot

9. indexOf() — subjectName

Objective

Objective: To understand and demonstrate the Java String method **indexOf()** using a simple example.

Task Description

Task Description: Write and execute the class **IndexOfDemo** to demonstrate that indexOf() finds the first position of a specified character or text.

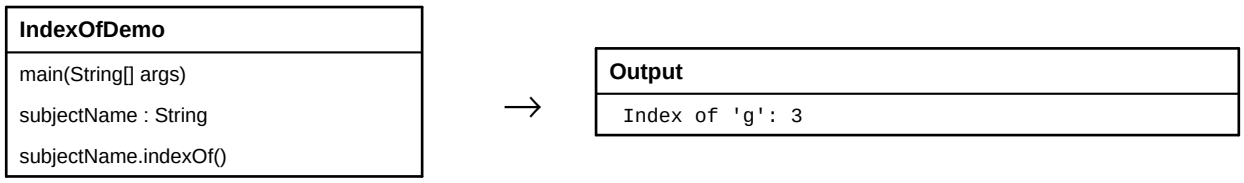
Step-wise Execution

1. Create subjectName.
2. Store "Programming".
3. Call indexOf('g').
4. Print the first matching index.

Algorithm / Logic

1. Start.
2. Initialize subjectName.
3. Search for the first occurrence of `g`.
4. Display its index.
5. Stop.

UML Diagram



Program Code

```
class IndexOfDemo {
    public static void main(String[] args) {
        String subjectName = "Programming";

        System.out.println("Index of 'g': "
                + subjectName.indexOf('g'));
    }
}
```

Code Explanation

1. `subjectName` stores "Programming".
2. `indexOf('g')` searches from the beginning for the first `g`.
3. String indexing begins at 0, so the first `g` is at index 3.

Output Screenshot

10. lastIndexOf() — languageName

Objective

Objective: To understand and demonstrate the Java String method **lastIndexOf()** using a simple example.

Task Description

Task Description: Write and execute the class **LastIndexDemo** to demonstrate that lastIndexOf() finds the last position of a specified character or text.

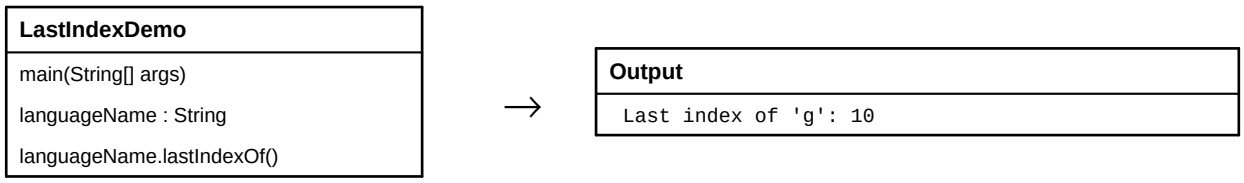
Step-wise Execution

1. Create languageName.
2. Store "Programming".
3. Call lastIndexOf('g').
4. Print the final matching index.

Algorithm / Logic

1. Start.
2. Initialize languageName.
3. Search from the end for `g`.
4. Display the last index.
5. Stop.

UML Diagram



Program Code

```
class LastIndexDemo {
    public static void main(String[] args) {
        String languageName = "Programming";

        System.out.println("Last index of 'g': "
                + languageName.lastIndexOf('g'));
    }
}
```

Code Explanation

1. `languageName` contains "Programming".
2. `lastIndexOf('g')` searches for the final occurrence of `g`.
3. The final `g` is at index 10, so the method returns 10.

Output Screenshot

11. replace() — technologyName

Objective

Objective: To understand and demonstrate the Java String method **replace()** using a simple example.

Task Description

Task Description: Write and execute the class **StringReplaceDemo** to demonstrate that replace() replaces characters or a sequence of characters with another sequence.

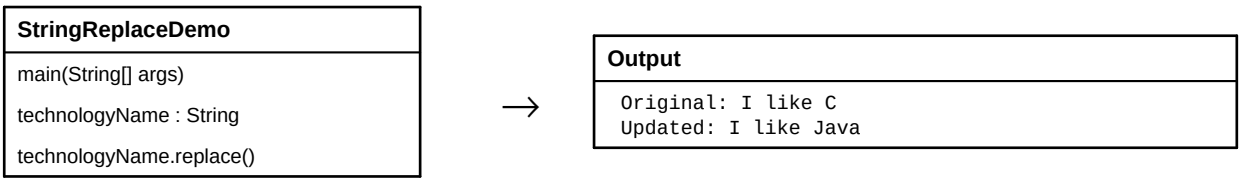
Step-wise Execution

1. Create technologyName.
2. Store "I like C".
3. Call replace("C", "Java").
4. Display the updated string.

Algorithm / Logic

1. Start.
2. Initialize technologyName.
3. Replace C with Java.
4. Print original and updated strings.
5. Stop.

UML Diagram



Program Code

```
class StringReplaceDemo {
    public static void main(String[] args) {
        String technologyName = "I like C";

        System.out.println("Original: " + technologyName);
        System.out.println("Updated: "
                + technologyName.replace("C", "Java"));
    }
}
```

Code Explanation

1. `technologyName` contains the text "I like C".
2. `replace("C", "Java")` changes the matching text C to Java.
3. The replaced string is returned and printed.

Output Screenshot

12. trim() — addressText

Objective

Objective: To understand and demonstrate the Java String method **trim()** using a simple example.

Task Description

Task Description: Write and execute the class **StringTrimDemo** to demonstrate that trim() removes leading and trailing whitespace from a string.

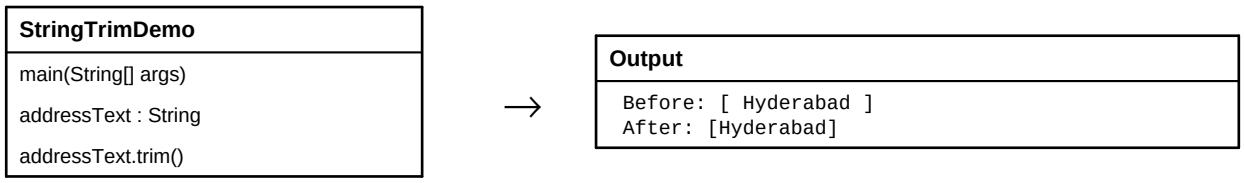
Step-wise Execution

1. Create addressText with leading and trailing spaces.
2. Print the original text inside brackets.
3. Call trim().
4. Print the cleaned text.

Algorithm / Logic

1. Start.
2. Initialize addressText with extra spaces.
3. Apply trim().
4. Display before and after values.
5. Stop.

UML Diagram



Program Code

```
class StringTrimDemo {
    public static void main(String[] args) {
        String addressText = "   Hyderabad   ";

        System.out.println("Before: [" + addressText + "]");
        System.out.println("After: [" + addressText.trim() + "]");
    }
}
```

Code Explanation

1. `addressText` contains spaces before and after "Hyderabad".
2. `trim()` removes whitespace from the beginning and end.
3. Spaces inside the text are not removed by trim().

Output Screenshot

13. startsWith() — websiteName

Objective

Objective: To understand and demonstrate the Java String method **startsWith()** using a simple example.

Task Description

Task Description: Write and execute the class **StartsWithDemo** to demonstrate that startsWith() checks whether a string begins with a specified prefix.

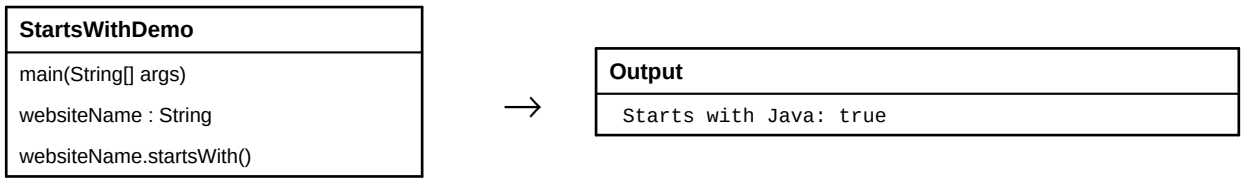
Step-wise Execution

1. Create websiteName.
2. Store "JavaProgramming".
3. Call startsWith("Java").
4. Print the Boolean result.

Algorithm / Logic

1. Start.
2. Initialize websiteName.
3. Check whether it starts with "Java".
4. Display true or false.
5. Stop.

UML Diagram



Program Code

```
class StartsWithDemo {
    public static void main(String[] args) {
        String websiteName = "JavaProgramming";

        System.out.println("Starts with Java: "
                + websiteName.startsWith("Java"));
    }
}
```

Code Explanation

1. `websiteName` contains "JavaProgramming".
2. `startsWith("Java")` checks the beginning of the string.
3. Because the string begins with Java, the result is `true`.

Output Screenshot

14. endsWith() — fileName

Objective

Objective: To understand and demonstrate the Java String method **endsWith()** using a simple example.

Task Description

Task Description: Write and execute the class **EndsWithDemo** to demonstrate that endsWith() checks whether a string ends with a specified suffix.

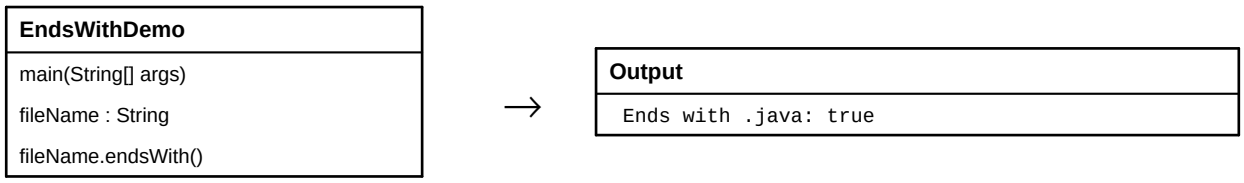
Step-wise Execution

1. Create fileName.
2. Store "Student.java".
3. Call endsWith(".java").
4. Print the Boolean result.

Algorithm / Logic

1. Start.
2. Initialize fileName.
3. Check the suffix ".java".
4. Display the result.
5. Stop.

UML Diagram



Program Code

```
class EndsWithDemo {
    public static void main(String[] args) {
        String fileName = "Student.java";

        System.out.println("Ends with .java: "
                + fileName.endsWith(".java"));
    }
}
```

Code Explanation

1. `fileName` stores "Student.java".
2. `endsWith(".java")` checks the ending portion of the filename.
3. The filename has the requested suffix, so the result is `true`.

Output Screenshot

15. concat() — greetingText

Objective

Objective: To understand and demonstrate the Java String method **concat()** using a simple example.

Task Description

Task Description: Write and execute the class **StringConcatDemo** to demonstrate that concat() joins two strings together.

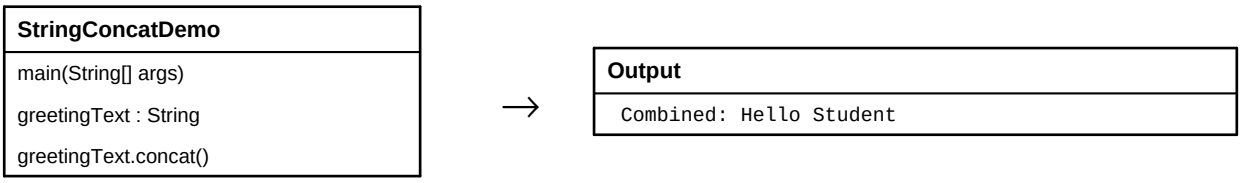
Step-wise Execution

1. Create greetingText and personName.
2. Assign values to both strings.
3. Call concat() using greetingText.
4. Print the combined string.

Algorithm / Logic

1. Start.
2. Initialize both strings.
3. Concatenate personName to greetingText.
4. Display the combined result.
5. Stop.

UML Diagram



Program Code

```
class StringConcatDemo {
    public static void main(String[] args) {
        String greetingText = "Hello ";
        String personName = "Student";

        System.out.println("Combined: "
                + greetingText.concat(personName));
    }
}
```

Code Explanation

1. `greetingText` contains "Hello " including a space.
2. `personName` contains "Student".
3. `greetingText.concat(personName)` joins both strings into one string.

Output Screenshot

15 String Methods — Name and Purpose

No.	String Method	Purpose
1	length()	Finds the number of characters in a string.
2	charAt()	Returns the character at a specified index position.
3	toUpperCase()	Converts all letters in a string to uppercase.
4	toLowerCase()	Converts all letters in a string to lowercase.
5	equals()	Compares two strings and returns true when their contents are equal.
6	equalsIgnoreCase()	Compares two strings without considering uppercase or lowercase differences.
7	contains()	Checks whether a specified sequence of characters exists in a string.
8	substring()	Extracts a part of a string using index positions.
9	indexOf()	Finds the first position of a specified character or text.
10	lastIndexOf()	Finds the last position of a specified character or text.
11	replace()	Replaces characters or a sequence of characters with another sequence.
12	trim()	Removes leading and trailing whitespace from a string.
13	startsWith()	Checks whether a string begins with a specified prefix.
14	endsWith()	Checks whether a string ends with a specified suffix.
15	concat()	Joins two strings together.